

DD_SUB-LM-01 - Subscription Lifecycle Management

Developer implementation guide: DD_SUB-LM-01-DEV_IMPL_GUIDE-v1.0.md.

1. Purpose

SUB-LM-01 defines the canonical Subscription entity for SOPHIX.

A Subscription is the long-lived operational record that says:

customer X consumes package Y at homepass Z under billing mode M

Billing, fulfillment, dunning, work orders, equipment, notifications, analytics, and customer care all anchor back to this record.

Use this DD when implementing:

- The subscription table.
- Subscription status catalog.
- Subscription creation in CREATED status.
- Internal state update APIs used by workflows.
- Active restriction catalog and active restriction storage.
- Cycle-window computation.
- Low-level subscription Kafka events.

2. Ownership

SUB-LM-01 owns:

- Subscription master entity.
- Subscription status catalog.
- Subscription transition reason catalog.
- Subscription restriction catalog.
- active_restrictions storage on the subscription.
- Cycle model fields stored on the subscription.
- Internal state mutation APIs used by workflow BPMNs.
- Generic low-level subscription events.

SUB-LM-01 does not own:

- Operation orchestration. See SUB-WF-FRAMEWORK-01 and operation DDs.
- Activation, pause, resume, terminate, upgrade, downgrade, relocation, migration, restrict, suspend-NP trigger APIs. Those are owned by operation DDs.
- Billing fees, proration money math, tax, invoices, wallets, or credit/debit notes. See BIL DDs.
- Network provisioning, restriction enforcement, suspension, termination, or HomePass claim/release. See FUL DDs.
- Package definition. See SIP-01.
- Service definition. See PLM-CFG-01.
- HomePass definition. See RLM-CFG-01.
- Customer/KYC data. See ILM-CFG-01.
- Equipment instance lifecycle. See OSR-INSTANCE-01.

3. Mental Model

SUB-LM-01 is the state-of-record layer. Workflows ask it to mutate state; it does not decide the full business process.

flowchart LR

```

FUL2[FUL-02 Order Capture] -->|POST /api/subscriptions| SUBLM[SUB-LM-01]
WF[SUB-WF operation BPMNs] -->|internal put-state / master-fields / restrictions| SUBLM
SUBLM -->|events after mutation| KAFKA[Kafka]
WF -->|billing intent| BIL01[BIL-01]
WF -->|network/fulfillment calls| FUL[FUL-03/04/05/07]
SUBLM -->|read package refs| SIP[SIP-01]
SUBLM -->|read homepass refs| RLM[RLM-CFG-01]

```

Workflow rule:

No downstream consumer should act on PENDING_* states as final business facts.

Operation BPMNs may temporarily put a subscription in PENDING_*. The final operation event is emitted by the operation DD only after the saga commits.

4. Core State Machine

Canonical state family:

```

CREATED
PENDING_ACTIVATION
ACTIVE
PENDING_PAUSE
SUSPENDED
PENDING_RESUME
PENDING_UPGRADE
PENDING_DOWNGRADE
PENDING_RELOCATION
PENDING_MIGRATION
PENDING_TERMINATION
TERMINATED
RETIRED

```

Restriction operations are special: adding/removing restrictions does not change status_code. It mutates active_restrictions.

State Meaning

Status	Meaning	Persistent?	Customer service active?
CREATED	Subscription row exists but service is not active.	Yes	No
PENDING_ACTIVATION	Activation saga in progress.	Transient	No
ACTIVE	Service is active and can be billed.	Yes	Yes
PENDING_PAUSE	Voluntary pause saga in progress.	Transient	Usually yes until commit
SUSPENDED	Service suspended by voluntary pause or non-payment.	Yes	No
PENDING_RESUME	Resume saga in progress.	Transient	No until commit
PENDING_UPGRADE	Package upgrade commit in progress.	Transient	Yes/commit-dependent
PENDING_DOWNGRADE	Package downgrade commit in progress.	Transient	Yes/commit-dependent
PENDING_RELOCATION	HomePass change commit in progress.	Transient	Commit-dependent
PENDING_MIGRATION	Package + HomePass change commit in progress.	Transient	Commit-dependent
PENDING_TERMINATION	Termination saga in progress.	Transient	Usually no after fulfillment step
TERMINATED	Contract/service ended.	Yes	No
RETIRED	Historical record retired from operational use.	Yes	No

5. Data Model

5.1 subscription

What this table is for:

subscription is the master operational record for an account's service. Every billing cycle, fulfillment action, dunning step, work order, equipment binding, and lifecycle workflow references this row.

SUB-LM follows the ILM-CFG-01 invariant: **Customer is identity; Account is operations**. account_id is therefore the operational anchor for the subscription. customer_id stays on the row as a denormalized identity/search reference so screens, integrations, and audit payloads do not need an extra customer lookup for every list view.

Use this table when:

- Creating a subscription after order capture.
- Reading the customer's current service/package/HomePass/status.
- Locking the subscription during a lifecycle workflow.
- Computing cycle windows for billing or workflow decisions.
- Recording current active restrictions.
- Auditing the current lifecycle state.

Do not use this table to store:

- Customer personal details. Use ILM-CFG-01.
- Package/service definitions. Use SIP-01 and PLM-CFG-01.
- Money amounts, invoices, balances, payments, or tax. Use BIL modules.
- Network endpoint topology. Use RLM-CFG-01/HomePass and FUL modules.

```
CREATE TABLE subscription (
  subscription_id      TEXT PRIMARY KEY,
  customer_id          TEXT NOT NULL,
  account_id           TEXT NOT NULL,
  operator_code        TEXT NOT NULL,
  homepass_id          TEXT NOT NULL,
  previous_homepass_id TEXT NULL,

  package_ref          TEXT NOT NULL,
  package_version_id   TEXT NULL,
  previous_package_ref  TEXT NULL,
  previous_package_version_id TEXT NULL,

  status_code          TEXT NOT NULL,
  billing_mode          TEXT NOT NULL,
  currency             TEXT NOT NULL,

  cycle_model          TEXT NOT NULL,
  cycle_anchor_day      INTEGER NULL,
  cycle_period_days     INTEGER NULL,
  cycle_frequency_months INTEGER NOT NULL DEFAULT 1,

  active_restrictions   JSONB NOT NULL DEFAULT '[]',

  current_transition_type TEXT NULL,
  current_transition_reason_code TEXT NULL,
  last_failure          JSONB NULL,

  activated_at          TIMESTAMPTZ NULL,
  suspended_at          TIMESTAMPTZ NULL,
  resumed_at            TIMESTAMPTZ NULL,
  terminated_at         TIMESTAMPTZ NULL,
  last_relocated_at     TIMESTAMPTZ NULL,
  last_migrated_at      TIMESTAMPTZ NULL,
  last_status_changed_at TIMESTAMPTZ NULL,

  start_date            DATE NULL,
  end_date              DATE NULL,

  created_at            TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  created_by            TEXT NOT NULL,
  updated_at            TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_by            TEXT NOT NULL,
  retired_at            TIMESTAMPTZ NULL,

  CHECK (billing_mode IN ('PREPAID','POSTPAID','PREPAYMENT')),
  CHECK (cycle_model IN ('CALENDAR_ANCHOR','ROLLING_DAYS')),
  CHECK (cycle_frequency_months > 0),
  CHECK (
    (cycle_model = 'CALENDAR_ANCHOR' AND cycle_anchor_day BETWEEN 1 AND 28 AND cycle_period_days IS NULL)
    OR
    (cycle_model = 'ROLLING_DAYS' AND cycle_period_days > 0 AND cycle_anchor_day IS NULL)
  )
);
```

Indexes:

```
CREATE INDEX idx_subscription_customer
  ON subscription (operator_code, customer_id, status_code);

CREATE INDEX idx_subscription_account
  ON subscription (operator_code, account_id, status_code);

CREATE INDEX idx_subscription_homepass
  ON subscription (operator_code, homepass_id, status_code);

CREATE INDEX idx_subscription_status
  ON subscription (operator_code, status_code);

CREATE UNIQUE INDEX uq_subscription_active_account_homepass_package
  ON subscription (account_id, homepass_id, package_ref)
  WHERE status_code NOT IN ('TERMINATED', 'RETIRED');
```

Column dictionary:

Column	Purpose	Notes
subscription_id	Primary identifier of the subscription.	Use ULID-style text, e.g. sub_01HZ. . .
customer_id	Customer identity that owns the account.	Denormalized reference to ILM-CFG-01 customer. No cross-module FK.
account_id	Operational account billed and serviced by this subscription.	Required reference to ILM-CFG-01 customer account. Billing, payments, work orders, equipment binding, and dunning use this as the operational anchor.
operator_code	Operator/country scope.	Examples: WIK, WUG, WTZ, YASSN.
homepass_id	Current service location / network termination point.	Reference to RLM-CFG-01 HomePass.
previous_homepass_id	Last HomePass before relocation/migration.	Null until the first move. Used for audit and rollback context.
package_ref	Current subscribed package code/ref.	Reference to SIP-01 Package.
package_version_id	Version of the package pinned to this subscription.	Set at activation and updated only by package-change/migration workflows.
previous_package_ref	Package before upgrade/downgrade/migration.	Null until first package change.
previous_package_version_id	Previous package version.	Used for audit, proration context, and support.
status_code	Current lifecycle status.	Must exist in subscription_status_code.
billing_mode	How the subscription is charged.	PREPAID, POSTPAID, or PREPAYMENT; immutable after activation.
currency	Currency used for this subscription.	Examples: KES, UGX, TZS, XOF.
cycle_model	Billing cycle model.	CALENDAR_ANCHOR or ROLLING_DAYS.
cycle_anchor_day	Day of month for calendar cycle.	Required for CALENDAR_ANCHOR; 1-28 only.
cycle_period_days	Fixed number of days for rolling cycle.	Required for ROLLING_DAYS.
cycle_frequency_months	How many months per cycle.	1 monthly, 3 quarterly, 6 semi-annual, 12 annual.
active_restrictions	Current partial-service restrictions.	JSON array. Empty array means no active restrictions.
current_transition_type	Operation currently affecting the subscription.	Examples: ACTIVATE, PAUSE, TERMINATE; null when stable.
current_transition_reason_code	Reason for the current/last transition.	References subscription_transition_reason.reason_code by convention.
last_failure	Structured details of last failed workflow mutation.	JSON object written by revert; null after clean success.
activated_at	Timestamp when service first became ACTIVE.	Written by activation workflow.
suspended_at	Last timestamp when subscription became SUSPENDED.	Written by pause or suspend-NP workflow.
resumed_at	Last timestamp when subscription returned to ACTIVE from SUSPENDED.	Written by resume workflow.
terminated_at	Timestamp when subscription became TERMINATED.	Written by termination workflow.
last_relocated_at	Last successful relocation timestamp.	Written by relocation workflow.
last_migrated_at	Last successful migration timestamp.	Written by migration workflow.
last_status_changed_at	Last time status_code changed.	Updated by internal state mutation APIs.

Column	Purpose	Notes
start_date	Contract/service start date.	Date-level business field, not necessarily exact activation timestamp.
end_date	Contract/service end date if fixed-term.	Null for open-ended subscriptions.
created_at, created_by	Creation audit.	Required on all rows.
updated_at, updated_by	Last update audit.	Required on all rows.
retired_at	Soft retirement timestamp.	Only for terminal historical records.

Sample row:

Column	Sample value	Meaning
subscription_id	sub_01J0WIKKAMAU001	Kamau's fiber subscription record.
customer_id	cust_01J0KAMAU001	Customer master record in ILM-CFG-01.
account_id	acc_01J0KAMAU_HOME	Kamau's home service account; a second holiday-home account would get a different subscription.
operator_code	WIK	Wananchi Kenya.
homepass_id	hp_01J0NRBKAREN073	Current HomePass in Karen, Nairobi.
previous_homepass_id	NULL	Customer has not relocated yet.
package_ref	pkg_wik_gpon_100m_res	Current 100 Mbps residential GPON package.
package_version_id	pkgver_2026_05_001	Package version pinned at activation.
status_code	ACTIVE	Subscription is currently delivering service.
billing_mode	POSTPAID	Customer is billed after usage/cycle close.
currency	KES	Billing currency.
cycle_model	CALENDAR_ANCHOR	Customer bills on a calendar day.
cycle_anchor_day	1	Cycle starts on day 1 of the month.
cycle_frequency_months	1	Monthly billing.
active_restrictions	[]	No partial restriction active.
current_transition_type	NULL	No operation in progress.
activated_at	2026-05-27T09:30:00+03:00	Service activated on May 27, 2026.
start_date	2026-05-27	Contract/service start date.

Sample row after dunning restriction:

Column	Sample value	Meaning
subscription_id	sub_01J0WIKKAMAU001	Same subscription.
status_code	ACTIVE	Subscription remains active.
active_restrictions	[{"code": "OUTGOING_VOICE_BARRED", "fulfillmentAction": "AAA_RESTRICT_OUTGOING_VOICE", "reasonCode": "DUNNING_LEVEL_2"}]	Voice outgoing is barred by BIL-04 dunning, but data/other services may remain active.

5.2 subscription_status_code

What this table is for:

subscription_status_code is the lifecycle status catalog. It defines the allowed values for subscription.status_code and the semantic flags other modules use to understand each status.

Use this table when:

- Validating that a status code is allowed.
- Displaying status labels in admin/customer-care UI.
- Determining whether a status is terminal, pending, active, or generally billable.
- Seeding operator/deployment-specific lifecycle status behavior.

Do not use this table to decide full operation eligibility. For example, whether a customer can pause, resume, relocate, or terminate is owned by the relevant workflow DD and Drools rules.

```
CREATE TABLE subscription_status_code (
  code TEXT PRIMARY KEY,
  display_name TEXT NOT NULL,
  description TEXT NULL,
  is_active BOOLEAN NOT NULL DEFAULT FALSE,
  is_billable BOOLEAN NOT NULL DEFAULT FALSE,
  is_terminal BOOLEAN NOT NULL DEFAULT FALSE,
  is_pending BOOLEAN NOT NULL DEFAULT FALSE,
  allows_package_change BOOLEAN NOT NULL DEFAULT FALSE,
  allows_address_move BOOLEAN NOT NULL DEFAULT FALSE,
  active BOOLEAN NOT NULL DEFAULT TRUE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  created_by TEXT NOT NULL,
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_by TEXT NOT NULL
);
```

Seed rows:

Code	is_active	is_billable	is_terminal	is_pending
CREATED	false	false	false	false
PENDING_ACTIVATION	false	false	false	true
ACTIVE	true	true	false	false
SUSPENDED	false	configurable	false	false
PENDING_TERMINATION	false	false	false	true
TERMINATED	false	false	true	false
RETIRED	false	false	true	false

`SUSPENDED.is_billable` depends on operator/workflow rules. BIL-03/BIL-04 must consult the relevant pause/suspension reason context rather than assume all suspended subscriptions behave the same.

Column dictionary:

Column	Purpose	Notes
code	Machine-readable status code.	Primary key. Uppercase snake-case.
display_name	Human-readable status label.	Used by BO/admin UI.
description	Longer explanation of the status.	Useful for admin help text.
is_active	Whether the service is considered active.	Used by operational filters.
is_billable	Whether normal cycle billing may apply.	Billing may still need reason-specific logic.
is_terminal	Whether this status ends normal lifecycle movement.	TERMINATED, RETIRED.
is_pending	Whether this is a workflow transient status.	PENDING statuses should not be treated as final business outcomes.
allows_package_change	Whether package-change workflows may start from this status.	Usually true only for ACTIVE.
allows_address_move	Whether relocation/migration may start from this status.	Usually true only for ACTIVE.
active	Whether this catalog row is usable for new transitions.	Retired statuses remain for history.
created_at, created_by, updated_at, updated_by	Audit columns.	Required.

Sample status rows:

code	Sample meaning
CREATED	Subscription exists but has not been activated. Created by FUL-02 order flow before activation.
ACTIVE	Customer is receiving service. Billing, dunning, upgrade/downgrade, relocation, and restriction flows can operate.
SUSPENDED	Service is stopped temporarily, either voluntary pause or non-payment. Resume workflow can operate.

code	Sample meaning
TERMINATED	Subscription is ended and cannot return to active state. Customer needs a new subscription to rejoin.

5.3 subscription_transition_reason

What this table is for:

subscription_transition_reason is the reason-code catalog for status and lifecycle changes. It gives every transition a structured business reason instead of free text.

Use this table when:

- A workflow needs to store why a transition happened.
- UI needs to show selectable reasons for admin actions.
- Reports need to group transitions by business cause.
- Downstream modules need a stable reason code in event payloads.

Do not use this table to model full workflow rules. It identifies the reason; it does not decide whether the action is allowed.

```
CREATE TABLE subscription_transition_reason (
  reason_code      TEXT PRIMARY KEY,
  transition_type   TEXT NOT NULL,
  display_name     TEXT NOT NULL,
  description       TEXT NULL,
  active           BOOLEAN NOT NULL DEFAULT TRUE,
  created_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  created_by       TEXT NOT NULL,
  updated_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_by       TEXT NOT NULL
);
```

Examples:

```
CUSTOMER_REQUESTED_PAUSE
ADMIN_PAUSE
DUNNING_SUSPENSION
CUSTOMER_REQUESTED_RESUME
CUSTOMER_REQUESTED_RELOCATION
CUSTOMER_MOVED_NO_COVERAGE
CONTRACT_EXPIRED
```

Column dictionary:

Column	Purpose	Notes
reason_code	Machine-readable reason.	Primary key, e.g. CUSTOMER_REQUESTED_PAUSE.
transition_type	Operation family that uses this reason.	Examples: PAUSE, RESUME, TERMINATE, SUSPEND_NP.
display_name	Human-readable label.	Used in BO/admin UI and reports.
description	Business explanation.	Should explain when this reason is valid.
active	Whether the reason can be selected for new transitions.	Inactive reasons remain valid for history.
created_at, created_by, updated_at, updated_by	Audit columns.	Required.

Sample reason rows:

reason_code	transition_type	Meaning
CUSTOMER_REQUESTED_PAUSE	PAUSE	Customer asked to pause service temporarily.
DUNNING_LEVEL_3_NON_PAYMENT	SUSPEND_NP	BIL-04 escalated unpaid debt to non-payment suspension.
CUSTOMER_MOVED_NO_COVERAGE	TERMINATE	Customer moved outside service footprint and requested termination.
CONTRACT_EXPIRED	TERMINATE	Fixed-term subscription reached contractual end.

5.4 subscription_restriction

What this table is for:

subscription_restriction is the catalog of partial-service restrictions that can be applied while the subscription remains ACTIVE. The catalog defines the restriction code and the fulfillment action FUL-04 must execute.

Use this table when:

- BIL-04 dunning applies a partial restriction before full suspension.
- A backoffice user applies or removes an operational restriction.
- FUL-04 needs to know which network action corresponds to a restriction.
- Customer-care UI needs to display active restriction labels.

Do not use this table to store which subscriptions are currently restricted. Current per-subscription restrictions live in subscription.active_restrictions.

```
CREATE TABLE subscription_restriction (  
  code TEXT PRIMARY KEY,  
  display_name TEXT NOT NULL,  
  description TEXT NULL,  
  fulfillment_action TEXT NOT NULL,  
  customer_visible BOOLEAN NOT NULL DEFAULT TRUE,  
  active BOOLEAN NOT NULL DEFAULT TRUE,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  created_by TEXT NOT NULL,  
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  updated_by TEXT NOT NULL  
);
```

Seed examples:

Code	Fulfillment action
OUTGOING_VOICE_BARRED	AAA_RESTRICT_OUTGOING_VOICE
OUTGOING_ALL_BARRED	AAA_RESTRICT_OUTGOING_ALL
DATA_THROTTLED_LOW	AAA_THROTTLE_DATA_LOW
DATA_THROTTLED_HIGH	AAA_THROTTLE_DATA_HIGH
PREMIUM_CONTENT_BARRED	AAA_BLOCK_PREMIUM_CONTENT

Column dictionary:

Column	Purpose	Notes
code	Machine-readable restriction code.	Primary key. Uppercase snake-case.
display_name	Human-readable restriction label.	Used by admin/customer care UI.
description	Business explanation.	Should describe exactly what service capability is limited.
fulfillment_action	Action code FUL-04 translates into network commands.	SUB-LM stores it; FUL-04 interprets it.
customer_visible	Whether customers can see this restriction label.	Some internal restrictions may be hidden from self-service.
active	Whether restriction can be newly applied.	Inactive restrictions may remain in history.
created_at, created_by, updated_at, updated_by	Audit columns.	Required.

Sample restriction rows:

code	display_name	fulfillment_action	Meaning
OUTGOING_VOICE_BARRED	Outgoing voice barred	AAA_RESTRICT_OUTGOING_VOICE	Customer cannot initiate outgoing voice calls.
DATA_THROTTLED_LOW	Data throttled - low	AAA_THROTTLE_DATA_LOW	Internet remains available but at a low speed profile.
PREMIUM_CONTENT_BARRED	Premium content barred	AAA_BLOCK_PREMIUM_CONTENT	IPTV premium bundles are blocked while base services may remain active.

5.5 Active Restriction JSON Shape

Stored in subscription.active_restrictions.

[


```

{
  "code": "OUTGOING_VOICE_BARRED",
  "fulfillmentAction": "AAA_RESTRICT_OUTGOING_VOICE",
  "addedAt": "2026-05-21T02:00:18+03:00",
  "addedBy": "bil04-svc-account",
  "reasonCode": "DUNNING_LEVEL_2",
  "dunningReference": "bil04-dunning-wave-2026-05-21-L2",
  "notes": "Voice outgoing barred until invoice INV_01HZ is paid"
}
]

```

Rules:

- One active entry per restriction code.
- Array is replaced atomically through internal API.
- All active restrictions are cleared when a subscription commits to SUSPENDED or TERMINATED.
- Restrictions are not automatically restored on resume.

6. Business Rules

6.1 Subscription Master Rules

Rule	Description
R-SUB-LM-S-1	A subscription is identified by subscription_id, a ULID-style text ID.
R-SUB-LM-S-2	Only one non-terminal subscription may exist for (account_id, homepass_id, package_ref). customer_id alone is not sufficient because one customer may operate multiple accounts.
R-SUB-LM-S-3	package_ref must resolve to an ACTIVE package at activation time.
R-SUB-LM-S-4	homepass_id must resolve to a sellable HomePass at activation time.
R-SUB-LM-S-5	billing_mode is set at activation and is immutable for the same subscription.
R-SUB-LM-S-6	Package version is pinned at activation. Later package publishes do not mutate existing subscriptions.
R-SUB-LM-S-7	Fulfillment reads current HomePass service endpoints from RLM-CFG-01; endpoints are not snapshotted on the subscription.
R-SUB-LM-S-8	Active/suspended subscriptions cannot be soft-deleted. They must terminate first.
R-SUB-LM-S-9	start_date must be on or before service start, and end_date, when set, must be after start_date.

6.2 Cycle Rules

Rule	Description
R-SUB-LM-CYC-1	CALENDAR_ANCHOR requires cycle_anchor_day from 1 to 28.
R-SUB-LM-CYC-2	ROLLING_DAYS requires cycle_period_days > 0.
R-SUB-LM-CYC-3	Days 29-31 are not valid anchors. Operator overflow policy is handled by activation workflow before commit.
R-SUB-LM-CYC-4	Cycle fields are immutable after activation unless a future billing-mode/cycle-change workflow explicitly owns the change.
R-SUB-LM-CYC-5	SUB-LM computes date windows only. BIL-01 computes money.

6.3 Status Catalog Rules

Rule	Description
R-SUB-LM-SC-1	Status code is uppercase snake-case and unique.
R-SUB-LM-SC-2	Status code is immutable once referenced.
R-SUB-LM-SC-3	Status cannot be retired while non-terminal subscriptions reference it.
R-SUB-LM-SC-4	Pending statuses are transient and not final downstream business facts.

6.4 Restriction Rules

Rule	Description
R-SUB-LM-RES-1	Restriction code is uppercase snake-case and unique.
R-SUB-LM-RES-2	Restriction code is immutable once ever applied.
R-SUB-LM-RES-3	Each restriction must declare one fulfillment_action.
R-SUB-LM-RES-4	Multiple restrictions may be active at the same time.
R-SUB-LM-RES-5	FUL-04 interprets fulfillment actions; SUB-LM only records active restrictions.
R-SUB-LM-RES-6	Suspension and termination clear active restrictions.

6.5 Event Rules

Rule	Description
R-SUB-LM-EVT-1	SUB-LM emits low-level events only after its own DB mutation commits.
R-SUB-LM-EVT-2	Operation-named events are emitted by operation workflow DDs after the saga commits.
R-SUB-LM-EVT-3	Event payloads include customAttributes for downstream extensibility.

7. REST APIs

SUB-LM has two API surfaces:

- Public read/create APIs.
- Internal workflow mutation APIs.

State-affecting business trigger endpoints are owned by workflow DDs, not SUB-LM.

7.1 Create Subscription

Used by FUL-02 or backoffice capture flows to create the subscription row in CREATED.

```
POST /api/subscriptions
Authorization: Bearer <jwt-or-service-token>
Content-Type: application/json
Idempotency-Key: create-sub-cust_01HX-hp_01HX-pkg_01HX
```

```
{
  "customerId": "cust_01HX",
  "accountId": "acc_01HX_HOME",
  "operatorCode": "WIK",
  "homepassId": "hp_01HX",
  "packageRef": "pkg_gpon_100m_res",
  "billingMode": "POSTPAID",
  "currency": "KES",
  "cycleModel": "CALENDAR_ANCHOR",
  "cycleAnchorDay": 1,
  "cycleFrequencyMonths": 1,
  "startDate": "2026-05-27"
}
```

Response:

```
{
  "subscriptionId": "sub_01HX",
  "statusCode": "CREATED",
  "operatorCode": "WIK",
  "customerId": "cust_01HX",
  "accountId": "acc_01HX_HOME",
  "homepassId": "hp_01HX",
  "packageRef": "pkg_gpon_100m_res"
}
```

Implementation rules:

- Must be idempotent.
- Does not activate service.
- Does not create wallet or invoice by itself.
- Activation is triggered separately through SUB-WF-ACTIVATE-01.

7.2 Read Subscription

```
GET /api/subscriptions/{subscription_id}
Authorization: Bearer <jwt-or-service-token>
```

Response:

```
{
  "subscriptionId": "sub_01HX",
  "statusCode": "ACTIVE",
  "billingMode": "POSTPAID",
  "cycleModel": "CALENDAR_ANCHOR",
  "cycleAnchorDay": 1,
  "cycleFrequencyMonths": 1,
  "activeRestrictions": [],
  "currentTransitionType": null,
  "currentTransitionReasonCode": null,
  "lastFailure": null
}
```

7.3 List Subscriptions

```
GET /api/subscriptions?operatorCode=WIK&accountId=acc_01HX_HOME&statusCode=ACTIVE&page=0&size=50
Authorization: Bearer <jwt>
```

Supported filters:

- operatorCode
- customerId
- accountId
- homepassId
- statusCode
- billingMode
- packageRef

7.4 Internal Put State

Called by workflow workers only.

```
POST /api/internal/subscriptions/{subscription_id}/put-state
X-Service-Token: <subscription-workflow-token>
Content-Type: application/json
```

```
{
  "targetStatusCode": "PENDING_TERMINATION",
  "transitionType": "TERMINATE",
  "reasonCode": "CUSTOMER_REQUESTED",
  "operationId": "subop_01HX",
  "metadata": {
    "bpmnProcessInstanceId": "pi_01HX"
  }
}
```

Response:

```
{
  "subscriptionId": "sub_01HX",
  "previousStatusCode": "ACTIVE",
  "targetStatusCode": "PENDING_TERMINATION",
  "changedAt": "2026-05-27T10:15:00Z",
  "lmEventId": "evt_01HX"
}
```

Rules:

- Must run in a DB transaction.
- Must lock the subscription row.
- Must validate legal transition for the calling operation.
- Must write outbox event in same transaction.

7.5 Internal Revert

```
POST /api/internal/subscriptions/{subscription_id}/revert
X-Service-Token: <subscription-workflow-token>
Content-Type: application/json
```

```
{
  "targetStatusCode": "ACTIVE",
  "operationId": "subop_01HX",
  "failureReason": {
    "operation": "TERMINATE",
    "category": "FULFILLMENT",
    "reasonCode": "TERMINATION_FULFILLMENT_FAILED",
    "reasonDetail": "AAA timeout",
    "actionable": true,
    "processInstanceId": "pi_01HX"
  }
}
```

7.6 Internal Put Master Fields

Used by upgrade/downgrade/relocation/migration/activation/termination workflows.

```
POST /api/internal/subscriptions/{subscription_id}/put-master-fields
X-Service-Token: <subscription-workflow-token>
Content-Type: application/json
```

```
{
  "operationId": "subop_01HX",
  "fields": {
    "homepassId": "hp_01HY",
    "previousHomepassId": "hp_01HX",
    "lastRelocatedAt": "2026-06-01T09:00:00+03:00"
  }
}
```

Rules:

- Only supplied fields are changed.
- Immutable fields must reject changes unless a future DD explicitly owns the operation.
- Must emit SubscriptionMasterUpdated.

7.7 Internal Put Active Restrictions

```
POST /api/internal/subscriptions/{subscription_id}/put-active-restrictions
X-Service-Token: <subscription-workflow-token>
Content-Type: application/json
```

```
{
  "operationId": "subop_01HX",
  "activeRestrictions": [
    {
      "code": "OUTGOING_VOICE_BARRED",
      "fulfillmentAction": "AAA_RESTRICT_OUTGOING_VOICE",
      "addedAt": "2026-05-21T02:00:18+03:00",
      "addedBy": "bil04-svc-account",
      "reasonCode": "DUNNING_LEVEL_2"
    }
  ]
}
```

7.8 Compute Cycle Window

```
POST /api/internal/subscriptions/{subscription_id}/compute-cycle-window
X-Service-Token: <billing-or-workflow-token>
Content-Type: application/json
```

```
{
  "operationDate": "2026-05-13T10:00:00+03:00"
}
```

Response:

```
{
  "currentCycleStart": "2026-05-01T00:00:00+03:00",
  "currentCycleEnd": "2026-06-01T00:00:00+03:00",
  "currentCycleTotalDays": 31,
  "daysElapsedInCycle": 12,
  "daysRemainingInCycle": 19,
  "cycleModel": "CALENDAR_ANCHOR",
  "cycleAnchorDay": 1,
}
```

```
    "cycleFrequencyMonths": 1
}
```

SUB-LM computes dates only. BIL-01 computes amounts.

8. Cycle Window Algorithm

Calendar anchor:

```
CycleWindow computeCalendarAnchor(LocalDateTime operationAt, int anchorDay, int frequencyMonths) {
    LocalDate anchorThisMonth = LocalDate.of(operationAt.getYear(), operationAt.getMonth(), anchorDay);
    LocalDate cycleStart = anchorThisMonth.isAfter(operationAt.toLocalDate())
        ? anchorThisMonth.minusMonths(frequencyMonths)
        : anchorThisMonth;
    LocalDate cycleEnd = cycleStart.plusMonths(frequencyMonths);
    return CycleWindow.of(cycleStart, cycleEnd, operationAt);
}
```

Rolling days:

```
CycleWindow computeRollingDays(Instant activatedAt, Instant operationAt, int periodDays) {
    long daysSinceActivation = Duration.between(activatedAt, operationAt).toDays();
    long cycleIndex = Math.floorDiv(daysSinceActivation, periodDays);
    Instant cycleStart = activatedAt.plus(cycleIndex * periodDays, ChronoUnit.DAYS);
    Instant cycleEnd = cycleStart.plus(periodDays, ChronoUnit.DAYS);
    return CycleWindow.of(cycleStart, cycleEnd, operationAt);
}
```

Implementation notes:

- Use operator timezone where the workflow/DD requires local cycle behavior.
- Store timestamps as TIMESTAMPTZ.
- Avoid month-end anchors by enforcing 1-28.

9. Kafka Events

SUB-LM emits low-level events only.

Event	Topic	Fires when
SubscriptionCreated	sophix.subscription.subscription.created	Subscription row inserted
SubscriptionStateChanged	sophix.subscription.subscription.state-changed	Internal put-state/revert changes status
SubscriptionMasterUpdated	sophix.subscription.subscription.master-updated	Master fields changed
SubscriptionRestrictionsChanged	sophix.subscription.subscription.restrictions-changed	Active restrictions replaced
SubscriptionStatusCodeCreated/Updated/Retired	sophix.subscription.statuscode.*	Status catalog admin changes
SubscriptionRestrictionCreated/Updated/Retired	sophix.subscription.restriction.*	Restriction catalog admin changes

High-level events like `SubscriptionActivated`, `SubscriptionPaused`, `SubscriptionTerminated`, `SubscriptionRelocated`, and `SubscriptionSuspendedForNonPayment` are emitted by their owning workflow DDs after saga commit.

Example:

```
{
  "eventId": "subscription:subscription:sub_01HX:evt_01HY",
  "eventType": "SubscriptionStateChanged",
  "aggregateId": "sub_01HX",
  "timestamp": "2026-05-27T10:15:00Z",
  "actor": "workflow:SUB-WF-TERMINATE-01",
  "correlationId": "corr_01HX",
  "payload": {
    "subscriptionId": "sub_01HX",
    "operatorCode": "WIK",
    "previousStatusCode": "ACTIVE",
    "targetStatusCode": "PENDING_TERMINATION",
    "currentTransitionType": "TERMINATE",
    "currentTransitionReasonCode": "CUSTOMER_REQUESTED",
    "operationId": "subop_01HX",
  }
}
```

```

    "customAttributes": {}
  }
}

```

10. Auth

API type	Auth
Customer/admin read APIs	JWT, roles declared by consuming channel
Create subscription from FUL-02/internal channels	Service token or authorized BO role
Internal workflow mutation APIs	X-Service-Token, caller identity must be subscription workflow service
Catalog admin APIs	JWT with catalog/subscription admin role

Internal APIs must not be exposed to public/customer networks.

11. Error Handling

Condition	HTTP status	Error code
Subscription not found	404	SUBSCRIPTION_NOT_FOUND
Invalid status transition	409	INVALID_SUBSCRIPTION_TRANSITION
Immutable field change attempted	409	IMMUTABLE_FIELD_CHANGE
Duplicate active subscription	409	DUPLICATE_ACTIVE_SUBSCRIPTION
Package not active	409	PACKAGE_NOT_ACTIVE
HomePass not sellable	409	HOMEPASS_NOT_SELLABLE
Invalid cycle fields	400	INVALID_CYCLE_MODEL
Restriction code unknown	400	UNKNOWN_RESTRICTION_CODE
Internal caller not authorized	403	SUBSCRIPTION_INTERNAL_FORBIDDEN

Recommended error body:

```

{
  "error": {
    "code": "INVALID_SUBSCRIPTION_TRANSITION",
    "message": "Cannot transition subscription from TERMINATED to ACTIVE.",
    "correlationId": "corr_01HX",
    "details": {
      "currentStatus": "TERMINATED",
      "targetStatus": "ACTIVE"
    }
  }
}

```

12. Implementation Notes

This section is written for the developer implementing the subscription module. The main idea is simple: every state mutation must be deterministic, locked, validated, written with its outbox event in the same transaction, and safe to retry.

12.1 Row Locking

All internal mutation APIs must lock the subscription row before reading or changing status/master fields/restrictions.

Why this matters:

Two workflow workers may run close together. Example: a dunning suspension and a customer-requested pause could hit the same subscription at nearly the same time. Without a row lock, both workers can read ACTIVE, both can think they are allowed to proceed, and the later commit can overwrite the earlier one. FOR UPDATE prevents that.

```

SELECT *
FROM subscription
WHERE subscription_id = :subscriptionId
FOR UPDATE;

```

Basic sequence for every mutation API:

1. Start DB transaction.
2. Read subscription using SELECT ... FOR UPDATE.

3. Validate the current state and requested change.
4. Apply the update.
5. Insert the outbox event.
6. Commit.

Pseudocode:

```
@Transactional
public PutStateResponse putState(String subscriptionId, PutStateRequest req) {
    Subscription sub = repository.findByIdForUpdate(subscriptionId)
        .orElseThrow(() -> notFound("SUBSCRIPTION_NOT_FOUND"));

    transitionValidator.validate(sub.statusCode(), req.targetStatusCode(), req.transitionType());

    String previousStatus = sub.statusCode();
    sub.setStatusCode(req.targetStatusCode());
    sub.setCurrentTransitionType(req.transitionType());
    sub.setCurrentTransitionReasonCode(req.reasonCode());
    sub.setLastStatusChangedAt(clock.now());
    sub.clearLastFailureIfSuccessfulPendingMove();

    repository.save(sub);

    outbox.insert(subscriptionStateChangedEvent(sub, previousStatus, req));

    return responseFrom(sub, previousStatus);
}
```

Important rule: do not call external services while holding the row lock. The workflow should call billing/fulfillment outside SUB-LM. SUB-LM should only update its own DB and outbox inside the transaction.

12.2 Outbox

Every mutation that emits an event must write to the outbox in the same transaction.

```
BEGIN;

UPDATE subscription
SET status_code = :targetStatus,
    current_transition_type = :transitionType,
    updated_at = NOW()
WHERE subscription_id = :subscriptionId;

INSERT INTO messaging.outbox_event (...);

COMMIT;
```

Why this matters:

If the subscription update commits but the event is not recorded, downstream modules never learn about the change. If the event is recorded but the update rolls back, downstream modules see a false event. Writing both in one transaction prevents both problems.

Do this:

```
@Transactional
public void commitState(...) {
    // 1. lock + update subscription
    // 2. insert messaging.outbox_event
    // 3. transaction commits both together
}
```

Do not do this:

```
public void commitState(...) {
    repository.save(subscription); // commits
    kafkaProducer.send(event);      // can fail after DB commit
}
```

Outbox event minimum fields:

Field	Value
event_id	Unique event ID, e.g. subscription:subscription:sub_01HX:evt_01HY.
topic	Kafka topic, e.g. sophix.subscription.subscription.state-changed.
message_key	Aggregate key, e.g. subscription:subscription:sub_01HX.

Field	Value
envelope	Full event envelope per FOUNDATION_KAFKA.md.

The outbox dispatcher publishes later. The API response must not wait for Kafka publish.

12.3 Idempotency

Create subscription must use idempotency.

Suggested table:

```
CREATE TABLE subscription_idempotency_key (
  operator_code TEXT NOT NULL,
  idempotency_key TEXT NOT NULL,
  request_hash TEXT NOT NULL,
  status TEXT NOT NULL CHECK (status IN ('PROCESSING', 'SUCCEEDED', 'FAILED')),
  response_json JSONB,
  error_code TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  PRIMARY KEY (operator_code, idempotency_key)
);
```

If the same key is reused with a different request hash, return 409
IDEMPOTENCY_KEY_REUSED_WITH_DIFFERENT_REQUEST.

The primary key is what prevents a second row from being inserted for the same (operator_code, idempotency_key). Do not rely only on an application-level check, because two HTTP requests can arrive at the same time.

How to implement create idempotency:

1. Require Idempotency-Key on POST /api/subscriptions.
2. Canonicalize the request body.
3. Hash the canonical request body.
4. Start a database transaction.
5. Try to insert a reservation row with status = 'PROCESSING'.
6. If the reservation insert succeeds, this request owns the creation.
7. If the insert fails because the primary key already exists, load the existing row:
 - Different request hash: return 409.
 - Same request hash and SUCCEEDED: return the stored response.
 - Same request hash and PROCESSING: return 409 or 425 with Retry-After, because another identical request is still running.
 - Same request hash and FAILED: either return the stored failure or allow retry only if the failure policy explicitly says it is retryable.
8. For the owner request, create the subscription, insert the outbox event, store the response JSON, set status to SUCCEEDED, and commit.

The reservation insert can use INSERT ... ON CONFLICT DO NOTHING. If zero rows are inserted, the code must immediately read the existing row and compare the hash.

Pseudocode:

```
@Transactional
public CreateSubscriptionResponse create(CreateSubscriptionRequest req, String idemKey) {
    String requestHash = hash(canonicalJson(req));

    boolean reserved = idemRepo.tryInsertProcessing(
        req.operatorCode(),
        idemKey,
        requestHash
    );

    if (!reserved) {
        IdempotencyRecord existing = idemRepo.findForUpdate(req.operatorCode(), idemKey);

        if (!existing.requestHash().equals(requestHash)) {
            throw conflict("IDEMPOTENCY_KEY_REUSED_WITH_DIFFERENT_REQUEST");
        }
    }
}
```



```

    }

    if (existing.status() == SUCCEEDED) {
        return json.fromJson(existing.responseJson(), CreateSubscriptionResponse.class);
    }

    if (existing.status() == PROCESSING) {
        throw retryLater("IDEMPOTENCY_REQUEST_IN_PROGRESS");
    }

    throw conflict("IDEMPOTENCY_PREVIOUS_REQUEST_FAILED");
}

Subscription sub = createSubscriptionRow(req);
outbox.insert(subscriptionCreatedEvent(sub));

CreateSubscriptionResponse response = responseFrom(sub);
idemRepo.markSucceeded(req.operatorCode(), idemKey, response);
return response;
}

```

Example SQL for the reservation:

```

INSERT INTO subscription_idempotency_key (
    operator_code,
    idempotency_key,
    request_hash,
    status
)
VALUES (:operatorCode, :idempotencyKey, :requestHash, 'PROCESSING')
ON CONFLICT (operator_code, idempotency_key) DO NOTHING;

```

If this statement inserts one row, continue with create. If it inserts zero rows, the key already exists; read that row and apply the hash/status rules above.

Do not make idempotency optional for create. Channels and mobile networks retry HTTP requests; without idempotency, the same customer/order may create duplicate subscriptions.

12.4 Validation Order

Use the same validation order in code so errors are predictable.

For POST /api/subscriptions:

1. Authenticate caller.
2. Validate required fields and enum values.
3. Validate idempotency key.
4. Validate package exists and is active through SIP-01 or cached package snapshot.
5. Validate HomePass exists and is sellable through RLM-CFG-01 or cached HomePass snapshot.
6. Validate cycle fields.
7. Check duplicate active subscription.
8. Insert subscription row in CREATED.
9. Insert SubscriptionCreated outbox event.
10. Store idempotency response.

For put -state:

1. Authenticate service token.
2. Validate request fields.
3. Start transaction.
4. Lock subscription row.
5. Validate current status allows requested target.
6. Update state fields.
7. Insert SubscriptionStateChanged outbox event.
8. Commit.

12.5 Transition Validation

Do not hardcode only one giant if block. Use a small transition validator that is easy to test.

Example transition map:

```
private static final Set<Transition> ALLOWED = Set.of(
    t("CREATED", "PENDING_ACTIVATION", "ACTIVATE"),
    t("PENDING_ACTIVATION", "ACTIVE", "ACTIVATE"),
    t("ACTIVE", "PENDING_PAUSE", "PAUSE"),
    t("PENDING_PAUSE", "SUSPENDED", "PAUSE"),
    t("SUSPENDED", "PENDING_RESUME", "RESUME"),
    t("PENDING_RESUME", "ACTIVE", "RESUME"),
    t("ACTIVE", "PENDING_TERMINATION", "TERMINATE"),
    t("PENDING_TERMINATION", "TERMINATED", "TERMINATE")
);
```

Validation:

```
void validate(String from, String to, String operation) {
    if (!ALLOWED.contains(t(from, to, operation))) {
        throw conflict("INVALID_SUBSCRIPTION_TRANSITION",
            Map.of("from", from, "to", to, "operation", operation));
    }
}
```

Operation DDs may add operation-specific rules before they call SUB-LM. SUB-LM still enforces the core lifecycle safety net.

12.6 Master Field Updates

put-master-fields must update only the fields provided in the request.

Example:

```
{
  "fields": {
    "homepassId": "hp_new",
    "previousHomepassId": "hp_old",
    "lastRelocatedAt": "2026-06-01T09:00:00+03:00"
  }
}
```

Expected behavior:

- Update homepass_id.
- Update previous_homepass_id.
- Update last_relocated_at.
- Leave package_ref, billing_mode, cycle fields, and other fields unchanged.

Reject attempts to change immutable fields:

```
{
  "fields": {
    "billingMode": "PREPAID"
  }
}
```

Response:

```
{
  "error": {
    "code": "IMMUTABLE_FIELD_CHANGE",
    "message": "billingMode cannot be changed on an existing subscription."
  }
}
```

12.7 Active Restriction Updates

Treat active_restrictions as a full replacement array, not a partial patch. The workflow worker computes the desired final list, then SUB-LM stores it atomically.

Why:

- ADD and REMOVE flows can be retried safely.
- SUB-LM does not need to understand every restriction operation.
- The row lock prevents lost updates.

Implementation steps:

1. Lock subscription row.
2. Validate every restriction code exists and is active in `subscription_restriction`.
3. Validate no duplicate code in the incoming array.
4. Replace `active_restrictions`.
5. Insert `SubscriptionRestrictionsChanged` event with previous and current arrays.
6. Commit.

When status commits to `SUSPENDED` or `TERMINATED`, clear `active_restrictions` in the same transaction as the status change unless the calling workflow has already done so.

12.8 Handling `last_failure`

`last_failure` is for UI and operations support. It tells a support/admin user why the latest workflow failed and whether they can act on it.

Example:

```
{
  "operation": "TERMINATE",
  "category": "FULFILLMENT",
  "reasonCode": "TERMINATION_FULFILLMENT_FAILED",
  "reasonDetail": "AAA timeout while removing service profile",
  "actionable": true,
  "processInstanceId": "pi_01HX",
  "failedAt": "2026-05-27T10:20:00Z"
}
```

Rules:

- `revert` writes `last_failure`.
- A later successful state-affecting operation clears `last_failure`.
- Do not put stack traces or secrets in `last_failure`.
- Keep it customer-safe enough for support tooling, but not necessarily customer-visible.

12.9 External Reads During Validation

Some create/activation validation needs package and HomePass data. SUB-LM may call SIP/RLM directly or use a Kafka-maintained cache, depending on the implementation.

Rules:

- Source of truth remains SIP/RLM.
- Cache misses must fall back to source APIs.
- If source API is unavailable and no valid cached data exists, fail safely with dependency error.
- Do not create a subscription against a package or HomePass that cannot be validated.

12.10 Transaction Boundaries

Keep transactions short.

Good transaction:

```
lock subscription -> validate -> update row -> insert outbox -> commit
```

Bad transaction:

```
lock subscription -> call billing -> call fulfillment -> update row -> commit
```

Billing and fulfillment calls belong in workflow workers, not inside SUB-LM transactions.

13. Infrastructure Recommendations

13.1 Deployment Unit Clarification

SUB-LM-01 is the Subscription Lifecycle Management service/module.

It owns:

- Subscription master create/read/list APIs.
- Internal subscription mutation APIs used by workflow workers.
- Subscription status, transition reason, and restriction catalogs.
- Cycle-window calculation.
- Low-level subscription events.

It does not own Camunda orchestration. That belongs to SUB-WF-FRAMEWORK-01.

Deployment options:

Option	When to use	Notes
Separate subscription-management-service	Cleaner ownership, easier scaling, more mature deployment.	SUB-LM has its own pods, routes, logs, metrics, and deployment pipeline.
Packaged with subscription-workflow-service	Budget-conscious start.	Same deployable app, but keep separate code packages, routes, DB migrations, and metrics labels.

Recommendation for this project:

Start medium-budget with SUB-LM and SUB-WF in the same application deployment if infrastructure budget is tight, but keep them logically separate so they can be split later without redesign.

13.2 On-Prem Medium Deployment

Recommended starting point:

Component	Count	Size per node/pod	Notes
SUB-LM API pods	2	2 vCPU, 4 GB RAM	Handles subscription create/read/list and internal mutation APIs.
SUB-LM outbox dispatcher pods	2	1 vCPU, 2 GB RAM	Publishes low-level SUB-LM events if this module owns its own outbox dispatcher.
SUB-WF operation pods	See SUB-WF-FRAMEWORK-01	2 vCPU, 4 GB RAM starting point	Workflow orchestration, operation status, cancel, and Camunda start.
PostgreSQL	Foundation DB sizing	6 nodes retained	DB is critical. Use the sizing from FOUNDATION_DB.
Kafka	Foundation Kafka sizing	4 nodes	Used for subscription events.
Redis/cache	Foundation cache sizing	Optional for read-side/reference cache	Never use cache as source of truth for subscription state.

Medium-budget packaging:

- One deployable application:
 - SUB-LM routes: /api/subscriptions, internal mutation APIs
 - SUB-WF routes: /api/subscription-operations
 - separate service packages/classes
 - separate database migrations by DD
 - separate metrics labels: module=SUB-LM and module=SUB-WF

- Scale initially:
 - 2 application pods
 - 2 outbox dispatcher pods or workers

If traffic grows, split into:

```
subscription-management-service
subscription-workflow-service
subscription-worker-service
```

13.3 OS Recommendation

Use the same OS baseline as the foundation bundle:

- Ubuntu Server 24.04 LTS, or
- Red Hat Enterprise Linux 9.

Required host settings:

- Enable NTP/chrony.
- Set timezone handling to UTC at the OS and database layer.
- Keep application display formatting separate from stored timestamps.
- Configure log rotation.
- Configure node-level disk alerts.

13.4 Recommended Partitions

For VM-based application hosts:

Mount	Suggested size	Purpose
/	40 GB	OS and base packages.
/var/log	50 GB	Application logs, request logs, workflow mutation troubleshooting.
/opt/sophix	20 GB	Application binaries/config if not fully containerized.
/var/lib/containerd or /var/lib/docker	80 GB	Container images and runtime layers.
/tmp	10 GB	Temporary files. Do not store uploads or exports here permanently.

If running on Kubernetes, apply equivalent node disk sizing and log retention. Do not let application logs fill the root partition.

13.5 AWS Deployment

Recommended AWS baseline:

- Run SUB-LM APIs on EKS or ECS Fargate.
- For limited budget, run SUB-LM and SUB-WF in one ECS service or one EKS deployment.
- Use Amazon RDS PostgreSQL or the managed PostgreSQL pattern selected in FOUNDATION_DB.
- Use Amazon MSK or the foundation Kafka deployment for events.
- Use ElastiCache only for reference/read-side caching, not subscription source-of-truth.
- Use CloudWatch metrics and alarms for API errors, mutation failures, DB latency, outbox backlog, and stale pending states.

Suggested AWS alarms:

Metric	Alarm condition
SUB-LM API 5xx rate	Above agreed threshold for 5 minutes.
DB connection pool saturation	Pool usage above 80 percent for 5 minutes.
Subscription mutation latency	p95 above agreed SLA.
Outbox pending age	Oldest pending event older than 5 minutes.
Outbox failed rows	Any row with status = FAILED.
Pending subscription state age	PENDING_* state older than operation timeout plus grace period.

14. Cross-DD Dependencies

Dependency	Direction	Contract
SUB-WF-FRAMEWORK-01	Workflows call SUB-LM	Operation rows, Camunda workers, idempotency, concurrency
Operation workflow DDs	Workflows call SUB-LM	Trigger APIs and business orchestration
SIP-01	SUB-LM reads package refs	Package status/version validation
PLM-CFG-01	Fulfillment/billing read service composition	Service endpoint requirements
RLM-CFG-01	SUB-LM/workflows read HomePass	Sellability and service endpoints
BIL-01	Workflows call billing before commit	Pay-first invariant, proration amounts

Dependency	Direction	Contract
BIL - 03	Consumes subscription events and cycle fields	Cycle scheduling
BIL - 04	Triggers restrict/suspend/resume/terminate workflows	Dunning consequences
FUL - 03/04/05/07	Workflows call fulfillment	Network and HomePass execution
NOT - 01	Consumes operation events	Customer/admin notifications
OSR - INSTANCE - 01	Activation/termination workflows bind/unbind equipment	Equipment lifecycle

15. Build Order

Implement in this order:

1. Base subscription schema and status/restriction catalogs.
2. Create/read/list APIs.
3. Internal mutation APIs.
4. Cycle-window computation.
5. Outbox event emission.
6. Admin catalog APIs.
7. Integration with SUB-WF-FRAMEWORK-01 workers.
8. Integration tests with first operation workflow, usually activation.

16. Test Checklist

- [] Create subscription succeeds in CREATED.
- [] Duplicate non-terminal (customer, homepass, package) is rejected.
- [] Terminal subscription does not block future resubscribe.
- [] Invalid cycle model is rejected.
- [] Calendar anchors 29-31 are rejected.
- [] put - state locks row and emits event.
- [] Invalid transition is rejected.
- [] revert restores prior status and stores last_failure.
- [] put-master-fields rejects immutable billing mode change.
- [] put-active-restrictions replaces array atomically.
- [] Suspension/termination clears active restrictions.
- [] Cycle-window calculation is deterministic.
- [] Outbox event is rolled back if DB transaction fails.
- [] Idempotency returns same response for same key/request.
- [] SUB-LM health and readiness endpoints expose DB dependency status.
- [] Outbox backlog metrics and alerts work.

17. Developer Checklist

- [] Do not add business operation orchestration to SUB-LM.
- [] Do not compute money/proration amounts in SUB-LM.
- [] Do not call network provisioning directly from SUB-LM.
- [] Keep internal mutation APIs service-token protected.

- [] Use row locks for every state mutation.
- [] Emit only low-level SUB-LM events here.
- [] Put operation-specific rules in the operation DD.
- [] Store external module references as IDs, not cross-module FKs.